Context

Act as an Automation Architect to design the framework for Playwright Automation with TestNG

Instructions

[MANDATORY] Ask me 3 multiple choice questions one by one

[MANDATORY] Once after answering the first question ,proceed with net

[MANDATORY] Generate Framework Structure First After completing all the questions

[STRICT] Proceed Step by Step for framework implementation

[STRICT] Up on my approval proceed with the next step

Excellent choice. We'll proceed with a:

- Hybrid Automation Framework
- Playwright + TestNG
- POM-based layered architecture
- Utilities, Reporting, API support, DB support, Retry handling, CI-ready setup

---

Question 3/3 — Reporting & CI/CD Preference

Choose your preferred reporting and pipeline integration setup:

A. Extent Reports + Jenkins
B. Allure Reports + GitHub Actions
C. Extent Reports + Azure DevOps
D. Allure + Jenkins + Docker Integration

c

Playwright Automation Framework Structure

Stack Chosen

- Java
- Playwright

- TestNG
- Hybrid Framework
- Extent Reports
- Azure DevOps Integration

Step 1 — Framework Architecture Design

```
playwright-testng-framework/
|
 ├── src/
|    ├── main/
|    |    ├── java/
|    |    |    ├── base/
|    |    |    |    ├── BaseTest.java
|    |    |    |    ├── PlaywrightFactory.java
|    |    |    |    └── BrowserManager.java
|    |    |    |
|    |    |    ├── pages/
|    |    |    |    ├── LoginPage.java
|    |    |    |    ├── DashboardPage.java
|    |    |    |    └── CommonPage.java
|    |    |    |
|    |    |    ├── utils/
|    |    |    |    ├── ConfigReader.java
|    |    |    |    ├── WaitUtils.java
|    |    |    |    ├── ScreenshotUtil.java
|    |    |    |    ├── ExcelUtil.java
|    |    |    |    ├── JsonUtil.java
|    |    |    |    ├── RetryAnalyzer.java
|    |    |    |    └── LoggerUtil.java
|    |    |    |
|    |    |    ├── reports/
|    |    |    |    ├── ExtentManager.java
|    |    |    |    └── ExtentTestManager.java
|    |    |    |
|    |    |    ├── listeners/
|    |    |    |    ├── TestListener.java
|    |    |    |    └── SuiteListener.java
```

```
|   |   |   |
|   |   |   ├── constants/
|   |   |   |   └── FrameworkConstants.java
|   |   |   |
|   |   |   ├── api/
|   |   |   |   └── ApiClient.java
|   |   |   |
|   |   |   └── database/
|   |   |       └── DBManager.java
|   |   |
|   |   └── resources/
|   |       ├── config/
|   |       |   ├── config.properties
|   |       |   └── environments/
|   |       |       ├── qa.properties
|   |       |       └── prod.properties
|   |       |
|   |       ├── testdata/
|   |       |   ├── testdata.xlsx
|   |       |   └── users.json
|   |       |
|   |       └── log4j2.xml
|   |
|   └── test/
|       ├── java/
|       |   ├── tests/
|       |   |   ├── LoginTest.java
|       |   |   ├── DashboardTest.java
|       |   |   └── SmokeTest.java
|       |   |
|       |   ├── runners/
|       |   |   └── TestRunner.java
|       |   |
|       |   └── dataproviders/
|       |       └── TestDataProvider.java
|       |
|       └── resources/
|           └── testng.xml
|
├── reports/
```

```
|
├── screenshots/
|
├── logs/
|
├── azure-pipelines.yml
|
├── pom.xml
|
├── README.md
|
└── .gitignore
```

Architecture Highlights

1. Base Layer

Responsible for:

- Browser initialization
- Playwright lifecycle
- Thread-safe execution
- Parallel execution support

---

2. Page Layer

Contains:

- UI locators
- Page actions
- Reusable workflows

Pattern Used:

- Page Object Model (POM)

---

3. Utility Layer

Reusable utilities:

- Wait handling
- Config management
- Screenshot capture
- Excel/JSON readers
- Retry mechanism

---

4. Reporting Layer

Integrated with:

- Extent Reports
- Screenshot embedding
- Failure tracking

---

5. Listener Layer

Handles:

- Test start/end
- Logging
- Retry execution
- Report updates

---

6. API Layer

Supports:

- API validation
- Token generation
- Hybrid UI + API flows

---

7. Database Layer

Supports:

- DB validation

- Backend verification
- Query execution

---

8. CI/CD Layer

Azure DevOps ready:

- Maven execution
- Report publishing
- Parallel execution support

Recommended Design Principles

Principle	Usage
SOLID	Maintainability
DRY	Reusability
ThreadLocal	Parallel execution
Factory Pattern	Browser management
Singleton	Reporting
POM	UI abstraction

Execution Flow

TestNG XML
↓
BaseTest
↓
PlaywrightFactory
↓
Browser Launch
↓
Page Classes
↓
Utilities + Reporting

↓
Extent Report

Step 2 (NEXT)

If you approve this architecture, next we will implement:

Maven Dependencies + pom.xml Setup

Including:

- Playwright
- TestNG
- Extent Reports
- Log4j
- Apache POI
- Jackson
- Maven Surefire Plugin

Please approve to proceed with Step 2.

yes approved

Step 2 — Maven Dependencies + pom.xml Setup

We will now create the Maven configuration required for the framework.

Purpose of pom.xml

The pom.xml manages:

- Project dependencies
- Build lifecycle
- Test execution
- Plugins
- Reporting integrations

Recommended Maven Dependencies

Dependency	Purpose
Playwright	Browser automation
TestNG	Test execution
Extent Reports	Reporting
Log4j2	Logging
Apache POI	Excel handling
Jackson	JSON parsing
Maven Surefire	Test execution
SLF4J	Logging abstraction

Complete pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.playwright.framework</groupId>
    <artifactId>playwright-testng-framework</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>

        <playwright.version>1.52.0</playwright.version>
        <testng.version>7.10.2</testng.version>
        <extent.version>5.1.2</extent.version>
        <log4j.version>2.23.1</log4j.version>
        <poi.version>5.2.5</poi.version>
        <jackson.version>2.17.1</jackson.version>
    </properties>
```

```
<dependencies>

    <!-- Playwright -->
    <dependency>
        <groupId>com.microsoft.playwright</groupId>
        <artifactId>playwright</artifactId>
        <version>${playwright.version}</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>${testng.version}</version>
        <scope>test</scope>
    </dependency>

    <!-- Extent Reports -->
    <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>${extent.version}</version>
    </dependency>

    <!-- Log4j -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-api</artifactId>
        <version>${log4j.version}</version>
    </dependency>

    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>${log4j.version}</version>
    </dependency>

    <!-- Apache POI -->
    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
```

```
      <version>${poi.version}</version>
    </dependency>

    <!-- Jackson -->
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>${jackson.version}</version>
    </dependency>

    <!-- SLF4J -->
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.13</version>
    </dependency>

  </dependencies>

  <build>
    <plugins>

      <!-- Maven Compiler Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
        <configuration>
          <source>17</source>
          <target>17</target>
        </configuration>
      </plugin>

      <!-- Maven Surefire Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>

        <configuration>
          <suiteXmlFiles>
```

```
            <suiteXmlFile>
              src/test/resources/testng.xml
            </suiteXmlFile>
          </suiteXmlFiles>

          <systemPropertyVariables>
            <environment>qa</environment>
            <browser>chromium</browser>
          </systemPropertyVariables>

        </configuration>
      </plugin>

    </plugins>
  </build>

</project>
```

Recommended Java Version

Java 17 (Recommended)

Reason:

- Stable with Playwright
- Better performance
- Long-term support

Recommended Maven Commands

Clean Build

```
mvn clean install
```

Run Tests

```
mvn test
```

Run Specific Suite

mvn test -DsuiteXmlFile=testng.xml

Dependency Purpose Architecture

Playwright → UI Automation
TestNG → Test Execution
Extent → Reporting
POI → Excel Data
Jackson → JSON Handling
Log4j → Logging
Surefire → Maven Execution

Best Practices Applied

Version Management

All versions centralized under:

<properties>

Environment Config

Dynamic execution support:

-Dbrowser=chromium
-Denvironment=qa

Scalability

Supports:

- Parallel execution
- CI/CD
- Docker
- Cross-browser testing

Step 3 (NEXT)

If approved, next we will implement:

Core Playwright Engine

Including:

- PlaywrightFactory.java
- BrowserManager.java
- BaseTest.java
- ThreadLocal support
- Browser lifecycle management
- Parallel execution design

Please approve to proceed with Step 3.